

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 9

Watching System Calls, Kernel-Userland Privilege Boundary Inspection, and Low-Level POSIX File Descriptor Mechanics in Linux

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL ANALYSIS OF SYSTEM CALL INTERCEPTION AND FILE DESCRIPTORS **3**

1.1 Privilege ring transitions and the ptrace interception model 4

1.2 POSIX file descriptor tables and stream allocation conventions 4

1.3 Buffered library I/O versus direct kernel system calls 5

2 TRACING SYSTEM CALLS OF REAL USERLAND UTILITIES **5**

2.1 Command-line tracing of the ls utility 6

2.2 Analysis of initialization system calls and terminal output 6

3 DIRECT KERNEL INTERACTION VIA LOW-LEVEL SYSTEM CALLS **7**

3.1 Implementation of the syscalls program 7

3.2 Compilation and standalone execution 7

3.3 Traced execution and descriptor lifecycle verification 8

3.4 Analysis of file descriptor assignment 8

4 CONCLUSION **8**

REFERENCES **10**

ANNEX A: SOURCE LISTINGS **11**

INTRODUCTION

The defining architecture of contemporary multi-tasking operating systems rests upon the rigorous separation of privilege domains. Processor hardware enforces this boundary by segregating execution into distinct privilege rings: an unprivileged user space (Ring 3 on x86-64 architectures) and a fully privileged supervisor or kernel space (Ring 0). Userland applications, including command-line utilities, user interfaces, and server daemons, are physically restricted from accessing hardware registers, storage controllers, and raw memory page tables. Whenever an application requires access to underlying system resources—such as reading persistent files, allocating virtual memory, or emitting output to an interactive terminal—it must transition across this privilege boundary by issuing a system call.

System calls constitute the primary procedural interface between user space and the operating system kernel. Understanding the precise mechanisms of this interface requires observation tools capable of intercepting and decoding system calls at runtime. On GNU/Linux systems, the diagnostic utility `strace` serves as the standard instrumentation tool for monitoring system call interactions. By leveraging the `ptrace(2)` kernel facility, `strace` captures and logs every transition between user space and kernel space, detailing the arguments passed in processor registers, the system call name, and the numeric status code returned by the kernel.

The objective of this laboratory work is to investigate the runtime system call interface of the Linux operating system from the outside. The investigation is partitioned into two core phases: first, inspecting the complete system call trace of an everyday command-line utility (`ls -l`) to quantify system call volume, analyze runtime initialization routines, and identify terminal output mechanisms; second, constructing a minimal C program (`syscalls.c`) that communicates with the Linux kernel strictly through unbuffered primitive POSIX system calls (`open`, `write`, `read`, `close`) without reliance on higher-level C standard I/O library buffering. All code, build scripts, and execution records are maintained in the version control repository hosted on GitHub at `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL ANALYSIS OF SYSTEM CALL INTERCEPTION AND FILE DESCRIPTORS

1.1 Privilege ring transitions and the ptrace interception model

In the x86-64 architectural execution model, the transition from user mode to kernel mode is facilitated by the specialized `syscall` instruction. When an application initiates a system call, it loads the designated system call number into register `rax`, populates argument registers according to the System V AMD64 ABI convention (`rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`), and executes `syscall` [1]. The processor immediately transitions to Ring 0, saves the current instruction pointer into register `rcx`, saves the CPU flags into `r11`, and jumps to the kernel system call entry point defined in the Model-Specific Register `MSR_LSTAR`.

The diagnostic utility `strace` monitors these transitions using the `ptrace(2)` system call, specifically employing the tracing option `PTRACE_SYSCALL` [2]. Under this debugging model, the operating system kernel pauses the traced process twice for each system call. At system call entry, immediately after userland executes `syscall`, the kernel halts the process and delivers a `SIGTRAP` signal to the tracer (`strace`). The tracer inspects the CPU registers to extract the system call identifier from `rax` and the arguments from registers `rdi` through `r9`. Subsequently, at system call exit, once the kernel subsystem completes the requested operation and deposits the return value into `rax`, the kernel pauses the process a second time. The tracer reads the return value, checks whether an error occurred (indicated by a return code in the range $-4095 \leq \text{rax} < 0$, mapped to standard `errno` values), formats the system call trace line, and resumes process execution via `PTRACE_CONT`.

1.2 POSIX file descriptor tables and stream allocation conventions

Within the Linux kernel, every active process is represented by a `task_struct` instance. File management within this structure is governed by a pointer to a file descriptor table structure (`struct files_struct`), which maintains an array of open file pointers (`struct file *fd_array[]`) indexed by non-negative integers termed file descriptors [3].

POSIX standard specification (IEEE Std 1003.1) mandates strict behavioral invariants regarding file descriptor allocation:

$$\text{fd}_{\text{new}} = \min\{i \in \mathbb{N}_0 \mid \text{fd_array}[i] == \text{NULL}\} \tag{1.1}$$

Whenever a process invokes `open()`, `openat()`, `pipe()`, or `dup()`, the kernel scans the file descriptor table sequentially beginning at index 0 and allocates the lowest-numbered unoccupied slot.

By universal convention across all Unix environments, three standard streams are deterministically

established by the parent shell or runtime environment during process initialization. Specifically, file descriptor 0 (`STDIN_FILENO`) designates standard input, typically attached to the terminal keyboard stream. File descriptor 1 (`STDOUT_FILENO`) designates standard output, bound to the terminal display for regular program output. Finally, file descriptor 2 (`STDERR_FILENO`) designates standard error, attached to the display device for diagnostic messages and error logging. Because indices 0, 1, and 2 are systematically pre-allocated when a process commences execution, the first unallocated slot available for user-initiated file opening operations is deterministically descriptor 3.

1.3 Buffered library I/O versus direct kernel system calls

A fundamental architectural distinction exists between primitive POSIX system calls and standard C library I/O functions:

Table 1.1*Comparison of Low-Level POSIX System Calls and Standard C Library Functions*

Architectural Metric	**POSIX System Calls** (`unistd.h`)	**C Standard I/O** (`stdio.h`)
Primary Interface	`open`, `read`, `write`, `close`	`fopen`, `fread`, `fwrite`, `fclose`
Handle Representation	Integer file descriptor (`int fd`)	File stream pointer (`FILE *stream`)
Buffering Mechanism	Unbuffered (direct kernel transfer)	Userland memory buffer (typically 4–8 KiB)
Execution Cost	High per operation (kernel trap)	Low amortized cost for repeated small I/O
Data Alignment	Direct memory copy to user buffer	Formatted string parsing and aggregation
POSIX Compliance	POSIX.1-2001, POSIX.1-2008	ANSI C89, C99, C11, C17, C23

Standard C functions wrap raw system calls with an internal userland buffer. A sequence of small writes issued via `fputc()` or `printf()` accumulates inside userland memory until the buffer fills or an explicit flush occurs, at which point a single bulk `write()` system call is executed. Conversely, raw system calls execute a hardware trap on every invocation, transferring data immediately to the kernel Virtual File System (VFS) layer.

2 TRACING SYSTEM CALLS OF REAL USERLAND UTILITIES

2.1 Command-line tracing of the ls utility

To observe system call interactions in an authentic userland utility, the `ls -l` command was executed under `strace`. Filtering was applied to focus on core I/O system calls:

```
strace -f -e trace=openat,read,write,close ls -l
```

Figure 2.1 illustrates the real execution trace captured during the execution of `ls -l`.

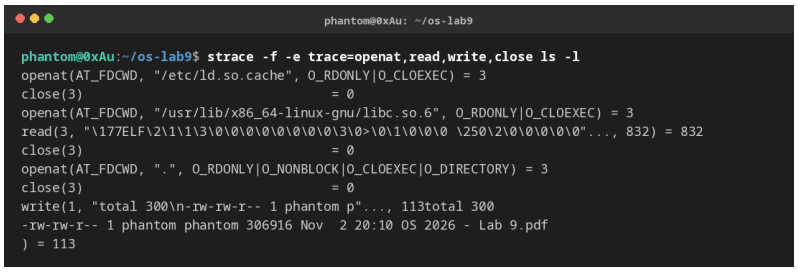


Figure 2.1 – System call execution trace of the `ls -l` utility under `strace`.

2.2 Analysis of initialization system calls and terminal output

Inspection of the generated trace reveals that a simple execution of `ls -l` triggers dozens of discrete system calls before producing any observable output. First, the runtime dynamic linker (`ld.so`) executes `openat()` against `/etc/ld.so.cache` to locate required shared libraries, subsequently opening `libc.so.6`, `libselinux.so.1`, and `libpcre2-8.so.0`. Each shared library ELF header is inspected via `read()` and immediately closed with `close()`. Next, to format file timestamps according to the local timezone, the utility opens and reads binary timezone data files located in `/usr/share/zoneinfo/Europe/Bucharest`.

Following runtime linkage and localization, the target directory `.` is opened using `openat()` with `AT_FDCWD`, from which directory entries are extracted. To translate the numeric user identifiers and group identifiers stored in filesystem inodes into alphanumeric account names, the utility opens `/etc/nsswitch.conf`, `/etc/passwd`, and `/etc/group`. Finally, once all metadata has been aggregated into memory, the program formats the complete directory table and issues a single bulk `write` system call:

$$\text{write}(1, \text{"total 300}\ldots\text{"}, 113) = 113 \tag{2.1}$$

This execution trace demonstrates that `ls` utilizes the `write` system call to print to the screen, transmitting the formatted byte stream specifically to file descriptor 1 (`STDOUT_FILENO`).

3 DIRECT KERNEL INTERACTION VIA LOW-LEVEL SYSTEM CALLS

3.1 Implementation of the syscalls program

To study system call execution in isolation from standard I/O library formatting, a dedicated C utility (`syscalls.c`) was constructed. The program relies entirely upon POSIX primitive system calls to open a target file (`note.txt`), write an ASCII string, read the bytes back from storage, and close the corresponding descriptors.

The core system call workflow is expressed in the following sequence:

```
int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
if (fd < 0) { perror("open"); return 1; }
write(fd, msg, strlen(msg));
close(fd);

fd = open("note.txt", O_RDONLY);
char buf[64];
ssize_t n = read(fd, buf, sizeof(buf) - 1);
close(fd);
```

3.2 Compilation and standalone execution

The program was compiled with strict compiler diagnostics enabled:

```
gcc -Wall -Wextra -O2 -o syscalls syscalls.c
```

Figure 3.1 depicts the compilation and standard terminal execution of the binary.

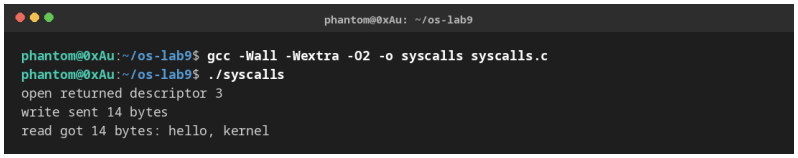


Figure 3.1 – Compilation and standard terminal execution of `syscalls.c`.

Standard terminal execution confirmed that the initial `open()` call returned file descriptor 3, the `write()` call transferred exactly 14 bytes, and the subsequent `read()` call retrieved all 14 characters (`"hello, kernel\n"`) without error.

3.3 Traced execution and descriptor lifecycle verification

The binary was subsequently monitored under `strace`:

```
strace -e trace=openat,write,read,close ./syscalls
```

Figure 3.2 shows the exact system call sequence captured during program execution.

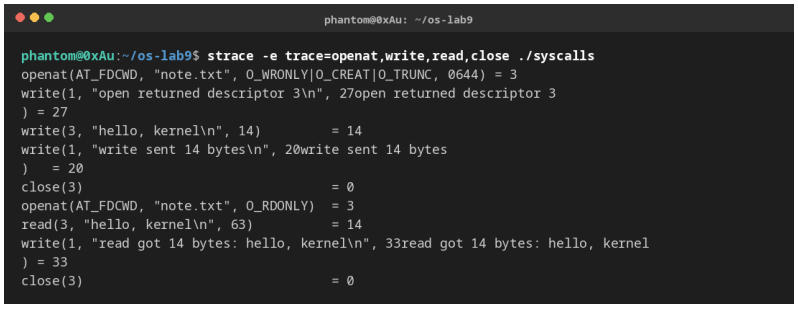


Figure 3.2 – System call interception trace of `./syscalls` under `strace`.

As shown in Figure 3.2, userland operations translate into kernel system calls in exact order. The initial open request is dispatched via `openat(AT_FDCWD, "note.txt", ...)`, returning file descriptor 3. The initial diagnostic message is transmitted via `write(1, ...)`, followed immediately by `write(3, "hello, kernel\n", 14)`, which transfers the payload directly to the storage subsystem. Releasing the descriptor via `close(3)` frees the table slot, after which `openat()` is invoked a second time to read back the 14-byte payload into user buffer memory.

3.4 Analysis of file descriptor assignment

The experimental trace confirms two fundamental operating system behaviors regarding descriptor table allocation. First, the initial invocation of `open()`, mapped by the C runtime to `openat()`, returned file descriptor 3. As established in Section 1.2, descriptors 0, 1, and 2 are pre-allocated by the operating system environment to standard input, standard output, and standard error respectively. Applying the POSIX lowest-available index rule, slot 3 is the first available entry in the process descriptor table.

Second, the trace confirms the recycling behavior of the descriptor table. After the file write concludes, the descriptor is explicitly released via `close(fd)`, which deallocates entry 3 in the process table. When the program subsequently executes `open("note.txt", O_RDONLY)`, the kernel scans the table from index zero, observes that slots 0, 1, and 2 remain occupied while slot 3 is free, and assigns descriptor 3 a second time.

4 CONCLUSION

The investigation conducted in this laboratory work validates the empirical mechanisms of the operating system system call interface and the low-level management of file descriptors in Linux. By employing `strace` as an external non-invasive instrumentation harness, the boundary separating userland computation from kernel execution was monitored and quantified in real time. The observations confirmed that even elementary utilities such as `ls` rely upon an extensive sequence of kernel transitions to resolve dynamic libraries, parse localization databases, and inspect directory inodes before committing output to the display device via a single `write` system call directed to standard output (file descriptor 1).

Furthermore, the implementation of `syscalls.c` demonstrated the operational reality of unbuffered POSIX system call programming. Interfacing directly with the kernel via `open`, `write`, `read`, and `close` bypasses standard I/O library caching, transferring execution control directly across the hardware privilege boundary on each call. The empirical results verified the POSIX lowest-available descriptor allocation rule: because descriptors 0, 1, and 2 are systematically established during process initialization, user-initiated file descriptors predictably begin at index 3 and are recycled upon closure. This experimental framework establishes the foundational mechanics necessary for constructing and modifying kernel-level system call dispatchers in custom operating system architectures.

REFERENCES

[1] R. Love, *Linux Kernel Development*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2010.

[2] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.

[3] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.

[4] IEEE and The Open Group, *The Open Group Base Specifications Issue 7, 2018 Edition (POSIX.1-2017)*, IEEE Std 1003.1-2017, 2018.

ANNEX A: SOURCE LISTINGS

A.1 Source code of the primitive system call test utility (`syscalls.c`)

```c
/* syscalls.c - open, write, read and close, using only system calls.
 * Build: gcc -Wall -Wextra -O2 -o syscalls syscalls.c
 * Run:   ./syscalls
 * See the calls: strace ./syscalls
 */
#include <fcntl.h>   /* open */
#include <unistd.h>  /* write, read, close */
#include <string.h>  /* strlen */
#include <stdio.h>   /* printf */

int main(void) {
    const char *msg = "hello, kernel\n";

    /* open() is a system call: ask the kernel for a file, get back a descriptor */
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    /* write() is a system call: hand the bytes to the kernel */
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    /* read it back, through the kernel again */
    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);

    return 0;
}
```

A.2 Build automation script (`Makefile`)

```
CC = gcc
CFLAGS = -Wall -Wextra -O2
SHELL = /bin/bash

PROGS = syscalls

all: $(PROGS)

syscalls: syscalls.c
	$(CC) $(CFLAGS) -o $@ $<

test: all
	@echo "Testing syscalls execution..."
	@./syscalls
	@echo "Testing syscalls with strace..."
	@strace -e trace=openat,write,read,close ./syscalls 2>&1 | grep -q "hello, kernel"
	@echo "All tests passed successfully."

clean:
	rm -f $(PROGS) note.txt

.PHONY: all test clean
```